Laporan Tugas Besar 2

IF2211 Strategi Algoritma

Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon DOM

Semester II Tahun 2025/2026

e621

Disusun oleh:

13524038	An-Dafa Anza Avansyah
13524053	Muhammad Haris Putra Sulastianto
13524059	Raymond Jonathan Dwi Putra J

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

25 April 2026

Daftar Isi

BAB 1: Deskripsi Tugas **3**

BAB 2: Landasan Teori **6**

2.1 Document Object Model (DOM) 6
2.2 CSS Selector 7
2.3 Algoritma Breadth-First Search (BFS) pada Graf 8
2.4 Algoritma Depth-First Search (DFS) pada Graf 9
2.5 Web Scraping dan Parsing 11

BAB 3: Aplikasi BFS dan DFS **13**

3.1 Representasi Pohon DOM pada Program 13
3.2 Mekanisme Pencocokan CSS Selector 13
3.3 Cara Kerja Algoritma BFS pada Program 14
3.4 Cara Kerja Algoritma DFS pada Program 15
3.5 Pseudocode Algoritma 16
3.5.1 Pseudocode BFS 16
3.5.2 Pseudocode DFS 17
3.5.3 Pseudocode Pencocokan Selector 17
3.5.4 Pseudocode Fungsi Kunjungan 18
3.6 Analisis Kompleksitas Waktu 18
3.6.1 Kompleksitas Waktu BFS 18
3.6.2 Kompleksitas Waktu DFS 18
3.6.3 Perbandingan Waktu BFS dan DFS 19
3.7 Analisis Kompleksitas Ruang (Memori) 19
3.7.1 Kompleksitas Ruang BFS 19
3.7.2 Kompleksitas Ruang DFS 20
3.7.3 Perbandingan Memori BFS dan DFS 20

BAB 4: Implementasi dan Pengujian **21**

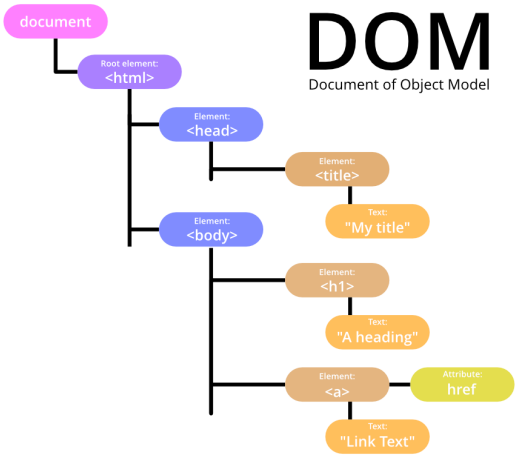
4.1 Lingkungan Pengembangan 21
4.2 Struktur Program dan Arsitektur 21
4.2.1 Struktur Repositori 21

4.2.2 Arsitektur Sistem 22
4.3 Implementasi Fitur Utama 23
4.3.1 Mekanisme Scraping dan Parsing HTML 23
4.3.2 Algoritma Traversal (BFS dan DFS) 29
4.3.3 Evaluasi CSS Selector 33
4.4 Implementasi Fitur Bonus 39
4.4.1 Visualisasi dan Animasi Penelusuran 39
4.4.2 Multithreading pada Traversal 40
4.4.3 Lowest Common Ancestor (LCA) Binary Lifting 42
4.5 Antarmuka Pengguna (UI) 42
4.6 Hasil Pengujian dan Analisis 44
4.6.1 Skenario Pengujian 44
4.6.2 Kasus Uji 1 (Mock HTML) 44
4.6.3 Kasus Uji 2 (Web Scrapping) 45
4.6.4 Analisis Performa BFS vs DFS 46

BAB 5: Kesimpulan dan Saran **49**
5.1 Kesimpulan 49
5.2 Saran 49

A Lampiran **51**
A.1 Tautan Repositori GitHub dan Video 51
A.2 Tabel Checklist Kelengkapan 51

BAB 1
DESKRIPSI TUGAS



Gambar 1: Struktur Pohon DOM

https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk *tree* yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti *parent*, *child*, dan *sibling* sehingga memudahkan pengolahan dokumen secara terprogram.

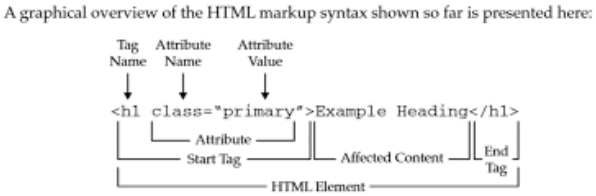
Struktur sebuah file HTML pada umumnya meliputi elemen *root* `<html>` yang berisi dua bagian utama yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata seperti judul halaman, *link stylesheet*, dan *script*, sedangkan `<body>` berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu *Breadth First Search* (BFS) dan *Depth First Search* (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan *selector* yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. **Tags**

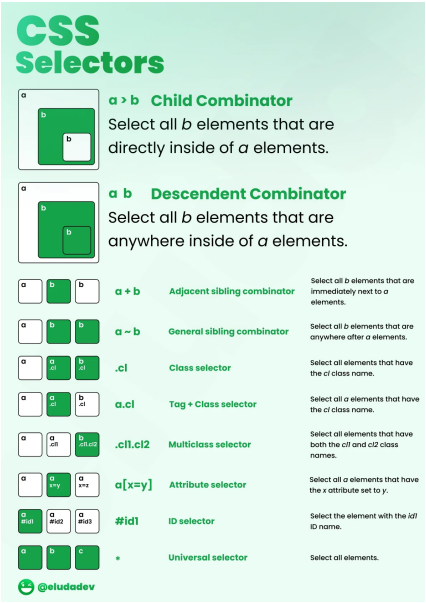


Gambar 2: Struktur Syntax HTML

https://tutorial.techaltum.com/htmlTags.html

Tag pada HTML merupakan penanda (*markup*) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu *browser* bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web. Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring `/` sebelum nama tag. Terdapat beberapa jenis tag HTML yang *self-closing*. Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai *node* pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti *parent*, *child*, dan *sibling*. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. **CSS Selector**



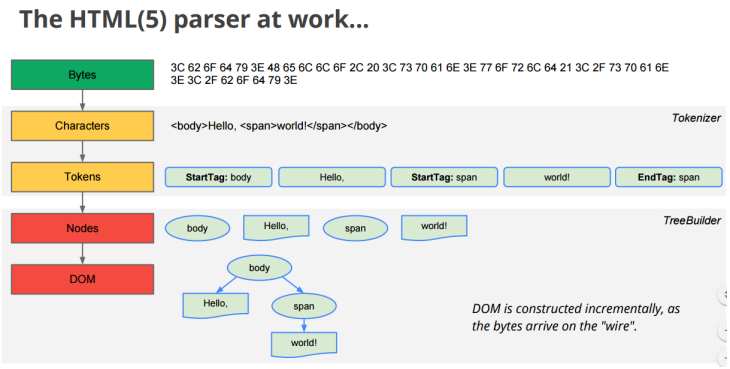
Gambar 3: Visualisasi CSS Selector

https://www.reddit.com//css_selectors_visually_explained/

CSS *Selector* adalah mekanisme pada CSS untuk memilih elemen HTML tertentu

pada struktur DOM agar dapat diberikan aturan *style*. Dengan *selector*, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa *selector* dasar yang umum digunakan adalah sebagai berikut: **Tag Selector** memilih elemen berdasarkan nama tag HTML (contoh: `p` memilih semua elemen `<p>`); **Class Selector** memilih elemen yang memiliki atribut *class* tertentu, ditulis dengan tanda titik (`.`) (contoh: `.box`); **ID Selector** memilih elemen dengan atribut *id* tertentu, ditulis dengan tanda pagar (`#`) (contoh: `#header`); dan **Universal Selector** memilih semua elemen HTML (contoh: `*`). *Combinator* merupakan cara untuk menggabungkan beberapa jenis *selector* dalam penentuan elemen yang akan terpengaruh, dan terdiri dari: *Child Combinator* (`>`), *Descendent Combinator* (spasi), *Adjacent Sibling Combinator* (`+`), dan *General Sibling Combinator* (`~`).

3. **HTML Parsing**



Gambar 4: Proses Parsing Pohon HTML

https://stackoverflow.com/how-does-a-parser-for-example-html-work

HTML *Parsing* adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (*tree*) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, *parser* memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki. Setiap tag HTML direpresentasikan sebagai *node* pada pohon. Tag yang berada di dalam tag lain akan menjadi *child node*, sedangkan tag yang membungkusnya menjadi *parent node*. *Node* paling atas pada struktur ini disebut *root*, yang umumnya merupakan elemen `<html>`. Dari *node root* tersebut, elemen-elemen lain seperti `<head>` dan `<body>` akan menjadi cabang yang membentuk struktur pohon DOM. Hasil dari proses *parsing* adalah *DOM Tree*, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma *Breadth First Search* (BFS) atau *Depth First Search* (DFS).

BAB 2
LANDASAN TEORI

2.1 Document Object Model (DOM)

Document Object Model (DOM) adalah antarmuka pemrograman untuk dokumen web. DOM merepresentasikan halaman sehingga program dapat mengubah struktur, gaya, dan konten dokumen, seperti HTML yang merepresentasikan halaman web. DOM merepresentasikan dokumen sebagai *node* dan objek sehingga bahasa pemrograman dapat berinteraksi dengan halaman tersebut.

Halaman web adalah dokumen yang dapat ditampilkan di jendela *browser* atau sebagai kode sumber HTML. Dengan DOM, elemen HTML seperti paragraf, gambar, atau tombol diubah menjadi objek JavaScript yang dapat diakses dan dimodifikasi. DOM ini sebagai 'jembatan' antara kode JavaScript dan tampilan halaman web dengan menyediakan model dokumen yang memungkinkan program mengubah struktur, konten, dan gaya halaman tanpa perlu me-*refresh* seluruh halaman.

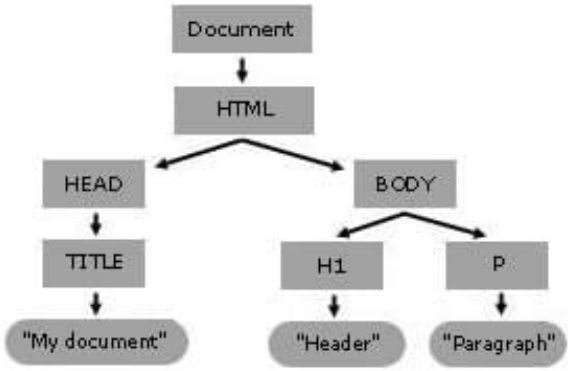
DOM dibangun menggunakan beberapa API yang bekerja bersama-sama. DOM inti mendefinisikan entitas yang menggambarkan setiap dokumen dan objek di dalamnya. Ini diperluas sesuai kebutuhan oleh API lain yang menambahkan fitur dan kemampuan baru ke DOM. Misalnya, API HTML DOM menambahkan dukungan untuk merepresentasikan dokumen HTML ke DOM inti, dan API SVG menambahkan dukungan untuk merepresentasikan dokumen SVG.

Pohon DOM adalah struktur pohon yang simpul-simpulnya mewakili isi dokumen HTML atau XML. Setiap dokumen HTML atau XML memiliki representasi pohon DOM. Misalnya, perhatikan dokumen berikut:

Struktur HTML

```html
<html lang="en">
  <head>
    <title>My Document</title>
  </head>
  <body>
    <h1>Header</h1>
    <p>Paragraph</p>
  </body>
</html>
```

Struktur DOM-nya terlihat seperti ini:



Gambar 5: Struktur DOM Hasil Parsing

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

Ketika peramban web menguraikan dokumen HTML, ia membangun pohon DOM dan kemudian menggunakannya untuk menampilkan dokumen tersebut.

2.2 CSS Selector

Cascading Style Sheets (CSS) *Selector* adalah sebuah mekanisme berupa pola string yang digunakan untuk mengidentifikasi, memilih, dan mencocokkan satu atau sekumpulan elemen (simpul/*node*) di dalam struktur pohon Document Object Model (DOM). Dalam konteks pengembangan web, *selector* umumnya digunakan untuk menerapkan aturan visual (*styling*) pada elemen HTML tertentu. Namun, dalam konteks komputasi dan *web scraping*, CSS *Selector* berfungsi sebagai kueri pencarian (*search query*) yang sangat kuat untuk mengekstraksi data spesifik dari sebuah graf atau pohon dokumen.

Secara konseptual, pencocokan CSS *Selector* beroperasi dengan cara mengevaluasi atribut dari setiap simpul pada pohon DOM dan memeriksa hubungan struktural antar simpul tersebut. CSS *Selector* dapat dibagi menjadi beberapa kategori dasar berdasarkan spesifisitas dan cara kerjanya, antara lain:

- **Universal Selector (`*`):** Memilih semua elemen di dalam dokumen DOM tanpa terkecuali. Ini setara dengan mengunjungi seluruh simpul dalam pohon tanpa memberikan filter kondisi pencocokan.
- **Tag/Type Selector:** Memilih elemen berdasarkan nama tag HTML aslinya. Misalnya, *selector* `div` akan mencocokkan semua simpul yang merupakan instansiasi dari tag `<div>`.
- **Class Selector (`.`):** Memilih elemen yang memiliki atribut kelas tertentu. Satu elemen HTML dapat memiliki beberapa kelas, dan banyak elemen dapat berbagi kelas yang sama. Penulisannya diawali dengan tanda titik, contohnya `.container` akan mencocokkan elemen seperti `<div class="container">`.
- **ID Selector (`#`):** Memilih elemen berdasarkan atribut *identifier* uniknya. Berdasarkan standar HTML, ID seharusnya bersifat unik di seluruh dokumen (hanya ada

satu elemen per ID). Penulisannya diawali dengan tanda pagar, contohnya `#header` akan mencocokkan `<nav id="header">`.

- **Attribute Selector (`[attr=value]`):** Memilih elemen berdasarkan keberadaan atribut tertentu atau nilai spesifik dari atribut tersebut, seperti `[href]` atau `[type="submit"]`.

Selain *selector* tunggal, CSS mendukung penggunaan **Kombinator (*Combinator*)** untuk mendefinisikan hubungan spasial dan hierarkis antar elemen di dalam pohon DOM. Hal ini sangat relevan dengan algoritma penelusuran graf (seperti BFS dan DFS) karena kombinator mendikte jalur traversal yang sah:

1. **Descendant Combinator (*Spasi*):** Dinyatakan dengan karakter spasi (contoh: `div p`). Kombinator ini memilih semua elemen `<p>` yang merupakan keturunan (baik *child* langsung maupun keturunan pada kedalaman tingkat berapa pun) dari elemen `<div>`.
2. **Child Combinator (`>`):** Dinyatakan dengan tanda lebih besar (contoh: `ul > li`). Berbeda dengan *descendant*, kombinator ini sangat ketat secara hierarki; ia hanya memilih elemen `<li>` yang merupakan anak langsung (*direct child* atau bertetangga langsung dengan *edge* derajat 1 ke bawah) dari `<ul>`.
3. **Adjacent Sibling Combinator (`+`):** Dinyatakan dengan tanda tambah (contoh: `h1 + p`). Memilih elemen `<p>` yang posisinya berada tepat setelah elemen `<h1>` dan memiliki *parent* (simpul induk) yang sama.
4. **General Sibling Combinator (`~`):** Dinyatakan dengan tanda tilde (contoh: `h1 ~ p`). Memilih semua elemen `<p>` yang muncul setelah elemen `<h1>` yang berada pada level hierarki (*parent*) yang sama, tidak peduli apakah letaknya bersebelahan langsung atau diselingi oleh simpul lain.

Pemahaman yang komprehensif terhadap pola struktural ini mutlak diperlukan dalam merancang mekanisme *Breadth-First Search* (BFS) maupun *Depth-First Search* (DFS), karena algoritma harus mampu menerjemahkan sintaks kueri ini menjadi aturan kondisional (*conditional logic*) saat berpindah dari satu simpul ke simpul lainnya di dalam pohon DOM.

2.3 Algoritma Breadth-First Search (BFS) pada Graf

Algoritma *Breadth-First Search* (BFS) adalah algoritma pencarian pada graf atau pohon (*tree*) yang dilakukan dengan cara mengunjungi simpul (*node*) secara melebar (per level). Pencarian dimulai dari simpul awal (biasanya simpul akar atau *root*) pada level n, kemudian mengeksplorasi seluruh simpul tetangga yang berada pada level yang sama secara berurutan, sebelum turun untuk mengunjungi simpul-simpul di level $n+1$ atau kedalaman selanjutnya. Karakteristik utama dari BFS adalah penelusuran yang memprioritaskan keluasan (*breadth*) tingkat hierarki daripada kedalaman (*depth*) cabang.

Dalam implementasinya, algoritma BFS beroperasi menggunakan struktur data antrean (*Queue*) yang mengedepankan prinsip *First-In-First-Out* (FIFO). Cara kerja sistematis algoritma ini adalah sebagai berikut:

1. Masukkan simpul awal (akar) ke dalam antrean.
2. Lakukan perulangan selama antrean tidak kosong: ambil (keluarkan) simpul dari elemen terdepan (*front*) antrean tersebut.
3. Evaluasi simpul yang baru saja dikeluarkan. Jika simpul tersebut merupakan target pencarian, maka solusi ditemukan dan proses pencarian dihentikan.
4. Jika simpul tersebut bukan merupakan solusi, masukkan (*enqueue*) seluruh simpul tetangga yang terhubung langsung dengan simpul tersebut ke bagian belakang antrean.

Penggunaan algoritma BFS sangat optimal untuk mencari lintasan terpendek (*shortest path*). Dalam konteks penelusuran graf, BFS akan memeriksa seluruh simpul tetangga secara merata sebelum menyusuri lintasan yang jaraknya lebih jauh dari simpul awal.

Contoh Penelusuran BFS pada Graf:
Misalkan terdapat sebuah pohon graf dengan simpul awal adalah **A** yang bertindak sebagai *root*. Simpul A terhubung ke simpul **B** dan **C**. Simpul B memiliki anak **D** dan **E**, sedangkan simpul C memiliki anak **F**. Terakhir, simpul E terhubung ke simpul **G**. Jika BFS digunakan untuk mencari simpul G dimulai dari simpul A, alur kunjungannya adalah sebagai berikut:

- **Level 0:** Algoritma mengevaluasi simpul **A**. Karena bukan target, tetangganya (B dan C) dimasukkan ke dalam antrean.
- **Level 1:** Mengambil **B** dari antrean (memasukkan D dan E), lalu mengambil **C** dari antrean (memasukkan F).
- **Level 2:** Mengambil **D** (tidak ada anak), lalu mengambil **E** (memasukkan G), dan mengambil **F** (tidak ada anak).
- **Level 3:** Mengambil **G** dari antrean. Target ditemukan dan pencarian selesai.

Urutan evaluasi simpul yang dikunjungi BFS secara keseluruhan adalah: **A** $\rightarrow$ **B** $\rightarrow$ **C** $\rightarrow$ **D** $\rightarrow$ **E** $\rightarrow$ **F** $\rightarrow$ **G**. Dapat dilihat bahwa graf ditelusuri secara berlapis, menyelesaikan satu kedalaman sebelum berpindah ke kedalaman berikutnya.

2.4 Algoritma Depth-First Search (DFS) pada Graf

Algoritma *Depth-First Search* (DFS) adalah algoritma pencarian pada graf atau pohon (*tree*) yang dilakukan dengan cara mengeksplorasi sedalam mungkin di sepanjang setiap cabang sebelum melakukan proses mundur (*backtracking*). Pencarian dimulai dari simpul

awal (*root*), lalu dilanjutkan dengan memprioritaskan kunjungan ke simpul anak (biasanya yang paling kiri atau berdasarkan prioritas tertentu) terus ke bawah hingga mencapai simpul daun (*leaf node*) terdalam atau simpul tak bertetangga. Karakteristik utama dari DFS adalah penelusuran yang memprioritaskan kedalaman (*depth*) lintasan daripada keluasan (*breadth*) level cabang.

Dalam implementasinya, algoritma DFS beroperasi menggunakan struktur data tumpukan (*Stack*) yang mengedepankan prinsip *Last-In-First-Out* (LIFO), atau diimplementasikan secara elegan menggunakan mekanisme rekursi. Cara kerja sistematis algoritma ini (versi iteratif dengan *Stack*) adalah sebagai berikut:

1. Masukkan simpul awal (akar) ke dalam tumpukan (*stack*).
2. Lakukan perulangan selama tumpukan tidak kosong: ambil (keluarkan) simpul dari elemen teratas (*top*) tumpukan tersebut.
3. Evaluasi simpul yang baru saja dikeluarkan. Jika simpul tersebut merupakan target pencarian, maka solusi ditemukan dan proses pencarian dihentikan.
4. Jika simpul tersebut bukan merupakan solusi, masukkan (*push*) seluruh simpul tetangga yang belum dikunjungi dari simpul tersebut ke bagian atas tumpukan.
5. Jika sebuah jalur telah dieksplorasi hingga ke ujung dan target belum ditemukan, algoritma akan *backtrack* (mundur) dan melanjutkan proses pencarian dari persimpangan cabang sebelumnya yang simpulnya masih tersimpan di dalam tumpukan.

Penggunaan algoritma DFS sangat optimal untuk mengeksplorasi graf yang memiliki kedalaman ekstrem atau untuk mencari keberadaan lintasan tertentu tanpa harus memperlebar pencarian di memori.

Contoh Penelusuran DFS pada Graf:
Menggunakan graf yang sama: simpul A terhubung ke B dan C; simpul B ke D dan E; simpul C ke F; dan simpul E ke G. Jika algoritma DFS digunakan untuk mencari simpul F dari simpul A (dengan asumsi penelusuran memprioritaskan cabang paling kiri terlebih dahulu), alur kunjungannya adalah sebagai berikut:

- **Langkah 1:** Mulai dari simpul **A**. Lanjutkan penelusuran langsung ke cabang kiri, yaitu simpul **B**.
- **Langkah 2:** Dari B, teruskan kembali ke cabang kiri sedalam mungkin, yaitu simpul **D**.
- **Langkah 3:** Simpul D adalah daun (tidak memiliki anak). Algoritma mundur (*backtrack*) ke simpul B, lalu menelusuri cabang B yang belum dikunjungi, yaitu simpul **E**.
- **Langkah 4:** Dari E, algoritma kembali turun menyusuri kedalaman menuju simpul **G**.

- **Langkah 5:** Simpul G adalah daun. Algoritma *backtrack* berulang kali: naik ke E, naik ke B, hingga kembali ke titik awal **A**.
- **Langkah 6:** Dari A, algoritma baru berpindah ke cabang kanannya yang belum tereksplorasi, yaitu simpul **C**. Dari C, turun ke simpul **F**. Target ditemukan.

Urutan evaluasi simpul yang dikunjungi DFS secara keseluruhan adalah: **A** → **B** → **D** → **E** → **G** → **C** → **F**. Dapat dilihat bahwa DFS menukik tajam secara vertikal hingga ke dasar suatu cabang, sebelum berbalik untuk mengeksplorasi cabang lainnya.

2.5 Web Scraping dan Parsing

Web scraping, atau ekstraksi data web, adalah sebuah proses pengambilan data mentah dari suatu situs web secara otomatis. Proses ini beroperasi dengan cara mengirimkan permintaan HTTP (*HTTP request*) ke sebuah URL tertentu. Sebagai respons atas permintaan tersebut, peladen (*server*) web akan mengembalikan konten halaman web yang umumnya berformat HTML. Konten HTML mentah ini memuat berbagai informasi berharga, seperti teks, gambar, tautan, hingga tabel data. Namun, data tersebut masih tertanam di dalam berbagai tag HTML sehingga sulit untuk diekstraksi dan dianalisis dalam bentuk mentahnya. Pendekatan *scraping* ini sangat krusial dalam mengumpulkan dataset berskala besar yang lazim digunakan untuk kebutuhan riset pasar, pemantauan harga, pengumpulan daftar properti, hingga pengumpulan artikel berita.

Dalam praktiknya, proses *data scraping* difasilitasi oleh berbagai pustaka dan kerangka kerja pemrograman untuk otomatisasi. Beberapa perangkat yang umum digunakan antara lain Scrapy, sebuah kerangka kerja yang fleksibel untuk membangun *web crawlers*; Selenium, yang mampu menyimulasikan interaksi pengguna pada *browser* sehingga sangat efektif untuk melakukan *scraping* pada situs web dinamis berbasis JavaScript; serta pustaka seperti Requests-HTML. Selain itu, terdapat pula berbagai platform *scraper* otomatis (seperti Bright Data, Octoparse, atau Diffbot) yang memungkinkan ekstraksi data tanpa penulisan kode tambahan.

Karena data hasil *scraping* umumnya berupa dokumen HTML yang tidak terstruktur, tahapan selanjutnya yang mutlak diperlukan adalah *Data Parsing* (penguraian data). *Parsing* merupakan proses analisis dan konversi data mentah tak terstruktur menjadi format struktur data yang lebih rapi dan siap untuk diproses lebih lanjut, seperti JSON, XML, atau CSV. Dalam konteks pengolahan web, *parsing* berfokus pada penguraian struktur dokumen HTML untuk mengekstraksi potongan informasi yang spesifik—misalnya mengambil teks dari tag judul (*title*), isi paragraf, atau atribut dari sebuah elemen.

Proses *parsing* tidak sekadar mengambil data, melainkan mengatur data tersebut berdasarkan hierarki pohon dokumen agar dapat disimpan di basis data atau dianalisis. Proses ini umumnya mengandalkan perangkat pengurai (*parser*) seperti BeautifulSoup pada bahasa Python, yang memungkinkan penelusuran struktur pohon HTML secara efisien untuk menemukan dan mengekstrak elemen yang bermakna. Di luar Python, terdapat pula pustaka seperti Jsoup pada Java, maupun pemanfaatan Ekspresi Reguler (*Regular Expressions*) untuk menangkap pola teks yang lebih kompleks dari data mentah. Secara

keseluruhan, kombinasi antara *web scraping* dan *HTML parsing* membentuk sebuah alur kerja integral dalam menangani data web, memproses data formulir, hingga mengolah respons API.

BAB 3
APLIKASI BFS DAN DFS

3.1 Representasi Pohon DOM pada Program

Sebelum algoritma penelusuran seperti BFS dan DFS dapat dioperasikan, dokumen HTML mentah harus diterjemahkan terlebih dahulu ke dalam bentuk struktur data pohon yang dapat dipahami oleh program. Pada aplikasi ini, setiap elemen HTML direpresentasikan secara individual sebagai sebuah objek bernama `DomNode`. Pemilihan representasi berbasis objek ini didasari oleh kebutuhan untuk menyimpan berbagai atribut yang melekat pada setiap elemen HTML secara rapi dan terstruktur.

Setiap objek `DomNode` menyimpan beberapa informasi esensial. Pertama, terdapat sebuah identifikator unik (`Id`) yang membedakan setiap simpul (*node*) di dalam pohon, misalnya `"node-1"` atau `"node-42"`. Kedua, nama tag HTML (`TagName`) yang mencerminkan jenis elemen aslinya, seperti `div`, `span`, atau `p`. Ketiga, atribut *id* HTML (`ElementId`) beserta daftar kelas CSS (`Classes`) yang nantinya akan menjadi kunci utama dalam proses pencocokan *selector*. Keempat, terdapat daftar simpul anak (`Children`) yang mendefinisikan hubungan hierarkis antar-elemen layaknya ranting pada sebuah pohon. Kelima, informasi kedalaman (`Depth`) yang menunjukkan seberapa jauh jarak simpul tersebut dari akar (*root*), di mana simpul akar selalu memiliki nilai kedalaman nol. Selain itu, setiap simpul juga dibekali dengan *array* untuk *binary lifting* (`Up`) berukuran 20, yang dipersiapkan untuk mendukung operasi kueri *Lowest Common Ancestor* (LCA) secara efisien. Adapun atribut `attributes` yang digunakan untuk menyimpan atribut *selector*.

Proses konversi dari teks HTML mentah menjadi pohon `DomNode` ini dilakukan oleh komponen *parser* yang melakukan pemrosesan secara manual terhadap string mentah kode HTML. Setiap kali sebuah tag ditemukan, program akan menginstansiasi objek `DomNode` baru dan merangkainya dengan simpul induk (*parent*) melalui daftar `Children`. Hasil akhir dari tahapan *parsing* ini adalah sebuah pohon DOM yang sepenuhnya merepresentasikan struktur dokumen situs web, siap untuk dijelajahi oleh algoritma pencarian.

3.2 Mekanisme Pencocokan CSS Selector

Sebagai jembatan antara penelusuran graf dan pencarian elemen, aplikasi membutuhkan sebuah mekanisme untuk menentukan apakah suatu simpul merupakan target yang sedang dicari, maka diperlukannya fungsi pengecekan `SelectorMatcher`, yang berfungsi mengevaluasi kecocokan antara sebuah simpul `DomNode` dengan CSS *Selector* yang dimasukkan oleh pengguna.

Mekanisme pencocokan ini bekerja dengan mendeteksi karakter awalan dari *string selector*. Jika *selector* diawali dengan karakter pagar (`#`), maka evaluasi difokuskan pada atribut *id*. Misalkan, `#header` akan mencari simpul yang nilai `ElementId`-nya adalah `"header"`. Jika diawali dengan karakter titik (`.`), maka pencocokan dilakukan pada atribut kelas, contoh `.container` akan menyaring simpul yang memuat `"container"` di dalam

daftar `Classes`-nya. Namun, apabila *selector* tidak memiliki karakter awalan khusus, program akan mencocokkannya langsung dengan nama tag HTML aslinya—seperti `div` yang akan menargetkan semua simpul bertag `<div>`. Agar lebih fleksibel, seluruh proses evaluasi ini dirancang bersifat *case-insensitive* (tidak membedakan huruf besar dan kecil), sehingga input seperti `DIV` maupun `div` akan memberikan hasil yang sama.

Komponen `SelectorMatcher` inilah yang akan selalu dipanggil oleh fungsi kunjungan (*visit function*) setiap kali algoritma BFS atau DFS singgah di sebuah simpul. Dengan pola arsitektur ini, alur kerja menjadi sangat terintegrasi: algoritma penelusuran mengambil peran dalam memandu *rute* perjalanan, sedangkan `SelectorMatcher` mengambil keputusan apakah simpul di ujung rute tersebut adalah data yang *valid*.

3.3 Cara Kerja Algoritma BFS pada Program

Penerapan *Breadth-First Search* (BFS) dalam aplikasi ini menerapakan konsep dasar dari BFS itu sendiri, namun juga memanfaatkan potensi *multithreading* yang ada pada *framework* .NET. Logika algoritma ini dibungkus di dalam kelas `BfsAlgorithm` melalui metode `ExecuteParallel`. Metode ini menerima empat parameter utama: simpul akar pohon DOM, fungsi aksi kunjungan, batas maksimal pencarian (opsional), serta koleksi data *thread-safe* untuk menampung elemen yang berhasil dicocokkan.

Penelusuran diinisiasi dengan memasukkan simpul akar ke dalam sebuah antrean bertipe `ConcurrentQueue`, yaitu varian struktur data antrean yang dijamin aman (*thread-safe*) meskipun diakses oleh banyak *thread* secara bersamaan. Setelah itu, algoritma akan masuk ke dalam putaran perulangan utama yang terus mengeksekusi data selama antrean tidak kosong dan target kuota hasil belum terpenuhi.

Pada setiap iterasinya, algoritma akan menghitung total simpul yang ada pada kedalaman level saat ini. Seluruh simpul tersebut kemudian dikeluarkan dari antrean secara serentak dan disimpan ke dalam sebuah daftar sementara. Tujuan pemisahan ini sangat strategis: agar seluruh simpul pada level yang sama dapat dieksekusi secara **paralel** menggunakan perintah `Parallel.ForEach`. Artinya, jika sebuah level pada pohon DOM memiliki ratusan elemen, beban pemeriksaannya dapat didistribusikan ke berbagai inti prosesor secara serentak.

Di dalam blok pemrosesan paralel tersebut, setiap *thread* bertugas menjalankan fungsi kunjungan pada simpul bagiannya. Fungsi ini akan mencatat riwayat kunjungan, mencocokkan *selector*, dan mengamankan data ke dalam daftar hasil jika kriteria terpenuhi. Terakhir, seluruh anak (*children*) dari simpul tersebut dimasukkan kembali ke antrean untuk dipersiapkan pada pemrosesan level berikutnya.

Untuk menjaga efisiensi, aplikasi ini juga mengadopsi mekanisme penghentian dini (*early termination*). Setiap kali fungsi kunjungan berjalan, program akan secara ketat memantau apakah kuota `maxResults` sudah tercapai. Jika sudah, pemrosesan paralel akan langsung diinterupsi menggunakan perintah `state.Stop()` dan putaran utama dihentikan melalui `break`. Fitur ini mencegah algoritma membuang-buang daya komputasi untuk menelusuri sisa pohon yang sudah tidak lagi relevan.

3.4 Cara Kerja Algoritma DFS pada Program

Penerapan algoritma *Depth-First Search* (DFS) dalam aplikasi ini, yaitu dengan menelusuri satu cabang hingga ke titik terdalamnya sebelum melangkah mundur dan mencoba percabangan lain. Implementasi algoritma ini diatur di dalam kelas `DfsAlgorithm` melalui metode `ExecuteParallel`.

Alih-alih menggunakan pemanggilan fungsi rekursif yang sering dicontohkan dalam buku teks, program ini membangun DFS secara iteratif dengan memanfaatkan `ConcurrentStack`. Keputusan arsitektural ini diambil dengan dua pertimbangan: pertama, untuk meminimalisasi risiko *stack overflow* yang rentan terjadi ketika menghadapi pohon DOM dengan tingkat kedalaman yang ekstrem; dan kedua, untuk memberikan ruang kontrol yang lebih eksplisit terhadap alur penelusuran.

Langkah pertama dimulai dengan menyisipkan simpul akar ke dalam tumpukan. Pada setiap putaran, simpul yang berada di posisi paling atas (*top*) akan dikeluarkan (*pop*) dan diproses. Proses pemeriksaan ini dieksekusi secara asinkron memanfaatkan `Parallel.Invoke`, yang memungkinkan fungsi kunjungan berjalan efisien tanpa harus membekukan alur utama program. Begitu simpul selesai diperiksa, seluruh anak dari simpul tersebut akan didorong masuk ke dalam tumpukan.

Ada satu detail teknis yang sangat krusial di sini: proses pendorongan (*push*) anak-anak simpul ke dalam tumpukan sengaja dilakukan secara **terbalik** (dari anak terakhir hingga anak pertama). Pendekatan ini merupakan konsekuensi logis dari sifat tumpukan yang berprinsip *Last-In-First-Out* (LIFO). Dengan menempatkan anak pertama di urutan terakhir, anak tersebut otomatis akan menduduki posisi puncak tumpukan dan dievaluasi lebih dulu pada putaran berikutnya. Trik matematis ini menjamin urutan penelusuran tetap mengalir dari kiri ke kanan (*left-to-right*), persis seperti perilaku standar DFS.

Berbeda dengan BFS yang memparalelkan eksekusi berdasarkan sekumpulan simpul dalam satu level, paralelisasi pada DFS bersifat lebih sempit. Pemanggilan `Parallel.Invoke` hanya mengisolasi eksekusi fungsi kunjungan pada satu simpul tunggal, sementara mekanisme perpindahan tumpukannya tetap berjalan secara berurutan. Meskipun percepatan komputasinya tidak sesignifikan BFS, strategi ini memastikan bahwa kaidah penelusuran kedalaman (*depth-first*) tidak rusak oleh perlombaan proses (*race condition*). Sama halnya dengan BFS, fitur terminasi dini juga diterapkan secara ketat pada DFS agar proses dapat langsung diakhiri saat target jumlah hasil telah tercapai.

3.5 Pseudocode Algoritma

3.5.1 Pseudocode BFS

Algorithm 1 BFS dengan Paralelisasi per Level pada Pohon DOM

```
procedure BFS-Parallel(root, visitAction, maxResults, matches)
    queue ← NewConcurrentQueue()
    queue.Enqueue(root)
    while queue tidak kosong do
        if maxResults ≠ null dan |matches| ≥ maxResults then
            break
        end if
        levelSize ← |queue|
        currentLevel ← daftar kosong
        for i ← 1 to levelSize do
            node ← queue.Dequeue()
            currentLevel.Add(node)
        end for
        for all node ∈ currentLevel secara paralel do          ▷ Parallel.ForEach
            if maxResults ≠ null dan |matches| ≥ maxResults then
                Hentikan pemrosesan paralel
                continue
            end if
            VisitAction(node)                    ▷ Catat kunjungan & cek selector
            if maxResults ≠ null dan |matches| ≥ maxResults then
                Hentikan pemrosesan paralel
            end if
            for all child ∈ node.Children do
                queue.Enqueue(child)
            end for
        end for
    end while
end procedure
```

3.5.2 Pseudocode DFS

Algorithm 2 DFS Iteratif dengan Stack pada Pohon DOM

```
procedure DFS-Parallel(root, visitAction, maxResults, matches)
    stack ← NewConcurrentStack()
    stack.Push(root)
    while stack tidak kosong do
        if maxResults ≠ null dan |matches| ≥ maxResults then
            break
        end if
        node ← stack.Pop()
        Jalankan secara paralel:                                  ▷ Parallel.Invoke
        if maxResults ≠ null dan |matches| ≥ maxResults then
            continue
        end if
        VisitAction(node)                          ▷ Catat kunjungan & cek selector
        if node.Children ≠ null then
            for i ← |node.Children| − 1 downto 0 do                ▷ Push terbalik
                stack.Push(node.Children[i])
            end for
        end if
    end while
end procedure
```

3.5.3 Pseudocode Pencocokan Selector

Algorithm 3 Pencocokan CSS Selector pada Simpul DOM

```
function Matches(node, selector)
    if selector kosong then
        return false
    end if
    if selector diawali '#' then
        return node.ElementId = selector[1..]                      ▷ ID Selector
    else if selector diawali '.' then
        targetClass ← selector[1..]
        return targetClass ∈ node.Classes                        ▷ Class Selector
    else
        return node.TagName = selector                            ▷ Tag Selector
    end if
end function
```

3.5.4 Pseudocode Fungsi Kunjungan

Algorithm 4 Fungsi Kunjungan Simpul (LogVisit)

```
procedure LogVisit(node)
    nodesVisited ← nodesVisited + 1                    ▷ Penambahan atomik
    maxDepth ← max(maxDepth, node.Depth)               ▷ Pembaruan atomik
    traversalPath.Enqueue(node.Id)
    traversalLog.Enqueue("Mengunjungi <" +node.TagName+ ">")
    if Matches(node, selector) then
        matchedNodes.Add(node)
        traversalLog.Enqueue("[MATCH] ditemukan")
    end if
end procedure
```

3.6 Analisis Kompleksitas Waktu

3.6.1 Kompleksitas Waktu BFS

Pada skenario terburuk (*worst case*), misalnya ketika elemen yang dicari sama sekali tidak ada atau tidak ada batasan kuota `maxResults`, algoritma BFS terpaksa harus mengunjungi seluruh n simpul. Pada setiap persinggahan, program melakukan operasi antrean (*enqueue* dan *dequeue*) yang memakan waktu konstan $O(1)$, serta mengevaluasi kecocokan *selector* yang secara praktis juga beroperasi pada kecepatan $O(1)$. Berdasarkan perhitungan tersebut, waktu eksekusi sekuensial (linier) dari BFS adalah:

$$T_{BFS}(n) = O(n)$$

Saat *multithreading* dilibatkan. Dengan memanfaatkan `Parallel.ForEach`, beban komputasi dapat didistribusikan. Jika sistem memiliki p buah *thread* yang siap bekerja dan sebuah level ke-i memuat n_i simpul, maka waktu penyelesaian level tersebut dapat dipangkas menjadi $O(n_i/p)$. Bila diakumulasikan secara ideal, waktu komputasinya menyusut menjadi:

$$T_{BFS\text{-}parallel} = O\left(\sum_{i=0}^{d} \frac{n_i}{p}\right) = O\left(\frac{n}{p}\right)$$

Di dunia nyata, rasio percepatan ini memang tidak murni linear karena akan selalu ada *overhead* sistem untuk sinkronisasi *thread*. Namun pada halaman web dengan struktur DOM yang kaya akan elemen sebaya (lebar), pemangkasan waktu ini tetap akan sangat terasa.

3.6.2 Kompleksitas Waktu DFS

Dalam perhitungan sekuensial, nasib DFS serupa dengan BFS. Algoritma akan melangkah satu per satu menyambangi n simpul pada skenario terburuk, melakukan operasi

tumpukan (*push* dan *pop*) dengan kompleksitas $O(1)$. Maka:

$$T_{DFS}(n) = O(n)$$

Meskipun DFS pada aplikasi ini juga diinjeksi dengan `Parallel.Invoke`, dampak percepatannya jauh lebih konservatif. Hal ini terjadi karena proses utama yang mengatur alur perjalanan di dalam tumpukan tetap berjalan seperti kendaraan di jalur tunggal (sekuensial). Paralelisasi di sini hanya berlaku pada durasi pemrosesan internal tiap simpul, bukan pada rute perjalanannya. Akibatnya, estimasi waktunya secara keseluruhan tetap berpijak pada kisaran:

$$T_{DFS\text{-}parallel} \approx O(n)$$

3.6.3 Perbandingan Waktu BFS dan DFS

Berdasarkan perbandingan waktu pada algoritma BFS dan DFS, algoritma BFS dan DFS memiliki kompleksitas waktu yang sama secara sekuensial, yaitu $O(n)$. Namun, pada saat algoritma dapat melakukan *multithreading*, algoritma BFS memiliki kompleksitas yang jauh lebih baik daripada algoritma DFS.

Aspek	**BFS**	**DFS**
Kompleksitas waktu (sekuensial)	$O(n)$	$O(n)$
Kompleksitas waktu (paralel)	$O(n/p)$	$\approx O(n)$
Efisiensi terminasi dini (elemen dangkal)	Tinggi	Rendah
Efisiensi terminasi dini (elemen dalam)	Rendah	Tinggi

Tabel 1: Perbandingan Kompleksitas Waktu BFS dan DFS

3.7 Analisis Kompleksitas Ruang (Memori)

3.7.1 Kompleksitas Ruang BFS

Arsitektur BFS mewajibkannya untuk menyimpan seluruh simpul dari level yang sedang aktif beserta calon anak-anaknya secara bersamaan di dalam antrean. Jumlah data ini berkorelasi langsung dengan seberapa "lebar" wujud pohon tersebut.

Pada kasus paling parah, di mana pohon berbentuk sangat rata dan melebar ke samping (dengan faktor percabangan b), level terbawah akan dijejali oleh $O(b^d)$ simpul. Kebutuhan memori BFS didefinisikan sebagai:

$$S_{BFS} = O(w)$$

di mana w mewakili rekor jumlah simpul terbanyak di satu level tertentu. Jika struktur DOM sangat dangkal namun berjejer panjang, nilai w bisa membengkak mendekati total elemen $O(n)$.

3.7.2 Kompleksitas Ruang DFS

Arsitektur DFS memiliki ikatan memori dengan tingkat kedalaman (*depth*) pohon. Tumpukan data pada DFS dirancang secara efisien untuk hanya "mengingat" jalan pulang dari simpul yang sedang ia pijak saat ini, beserta beberapa pintu persimpangan (*sibling*) yang belum sempat ia buka.

Jika direpresentasikan dalam rumus, untuk pohon sedalam d dengan faktor cabang b, tumpukan DFS hanya akan menanggung beban sekitar $b - 1$ simpul simpangan di setiap lapisan yang dilewatinya. Alhasil, batas maksimum memorinya adalah:

$$S_{DFS} = O(b \cdot d)$$

3.7.3 Perbandingan Memori BFS dan DFS

Berdasarkan perbandingan memori pada algoritma BFS dan DFS, arsitektur BFS memiliki kompleksitas memori yang sangat besar, yaitu mendekati total elemen $O(n)$. Sedangkan arsitektur DFS memiliki kompleksitas memori yang jauh lebih efisien, yaitu hingga tingkat kedalaman pohon $O(b \cdot d)$ dengan pohon sedalam d dengan faktor cabang b.

Aspek	**BFS**	**DFS**
Struktur data utama	`ConcurrentQueue`	`ConcurrentStack`
Kompleksitas ruang (umum)	$O(w)$	$O(b \cdot d)$
Kompleksitas ruang (terburuk)	$O(n)$	$O(n)$
Pohon lebar & dangkal	Boros memori	Hemat memori
Pohon sempit & dalam	Hemat memori	Hemat memori
Pohon DOM tipikal	Moderat	Hemat

Tabel 2: Perbandingan Kompleksitas Ruang BFS dan DFS

BAB 4
IMPLEMENTASI DAN PENGUJIAN

4.1 Lingkungan Pengembangan

Proyek ini dikembangkan dengan membaginya menjadi dua komponen utama, yaitu *frontend* dan *backend*. Komponen *frontend* bertugas untuk menyajikan antarmuka pengguna, menerima masukan berupa tautan (URL) maupun kode sumber HTML, serta menampilkan visualisasi pohon DOM dari hasil masukan tersebut.

Sementara itu, komponen *backend* bertugas memproses permintaan dari pengguna. Jika pengguna memberikan tautan, *backend* akan melakukan *scraping* untuk mengambil kode sumber HTML. Kode tersebut kemudian melewati proses *parsing* untuk diubah menjadi struktur pohon DOM. Selanjutnya, *backend* juga mengeksekusi algoritma pencarian elemen berdasarkan CSS *selector*, baik menggunakan pendekatan *Breadth-First Search* (BFS) maupun *Depth-First Search* (DFS).

Secara teknis, *frontend* dibangun menggunakan *framework* React yang berbasis JavaScript. Di sisi lain, *backend* dikembangkan menggunakan bahasa pemrograman C# di atas *framework* .NET dari Microsoft.

4.2 Struktur Program dan Arsitektur

Program ini terbagi menjadi dua komponen utama, yaitu backend yang menangani proses scraping, parsing, dan traversal DOM, serta frontend yang menyediakan antarmuka pengguna berbasis React.

4.2.1 Struktur Repositori

Program ini diorganisasikan ke dalam dua direktori utama di dalam folder `src`, yaitu `frontEnd` dan `backEnd`, serta menyertakan konfigurasi Docker.

```
+-- doc/                            # Dokumen laporan dan spesifikasi
+-- src/
    +-- backEnd/
    |   +-- algorithms/             # Implementasi algoritma utama
    |   |   +-- BfsAlgorithm.cs
    |   |   +-- DfsAlgorithm.cs
    |   |   +-- SelectorMatcher.cs
    |   +-- controllers/
    |   |   +-- DomController.cs    # Endpoint API untuk menerima request
    |   +-- models/                 # Struktur data
    |   +-- services/               # Logika pemrosesan (DOM Tree, web scraping,
    dll)
    |   |   +-- DomParserService.cs
```

```
    |   |   +-- HTMLProviderService.cs
    |   |   +-- TraversalService.cs
    |   +-- Dockerfile              # Konfigurasi container backend
    |   +-- Program.cs              # Entry point backend
    +-- frontEnd/
    |   +-- src/
    |   |   +-- components/         # Komponen visualisasi
    |   |   |   +-- InputForm.jsx
    |   |   |   +-- ResultsPanel.jsx
    |   |   |   +-- TreeView.jsx
    |   |   +-- services/
    |   |   |   +-- api.js          # Jembatan komunikasi ke endpoint backend
    |   |   +-- App.jsx             # Komponen utama halaman
    |   |   +-- main.jsx            # Entry point frontend
    |   +-- Dockerfile              # Konfigurasi container frontend
    +-- compose.yaml
```

4.2.2 Arsitektur Sistem

Aplikasi ini mengadopsi arsitektur *Client-Server* yang memisahkan tanggung jawab visualisasi dan komputasi. Keseluruhan sistem juga telah dikontainerisasi menggunakan **Docker** melalui `compose.yaml` untuk memudahkan *deployment*. Alur kerja sistem dirancang sebagai berikut:

1. **Client Layer (Frontend):** Pengguna berinteraksi melalui komponen `InputForm.jsx` untuk memasukkan tautan situs, parameter pencarian, serta jenis algoritma. Modul `api.js` kemudian akan membungkus data tersebut dan mengirimkan *HTTP POST request* ke server.

2. **Controller Layer (API):** *Request* dari *client* diterima oleh `DomController.cs`. Kontroler ini bertugas memvalidasi *input* (seperti memastikan URL valid atau teks HTML tidak kosong) dan mendistribusikan tugas ke layanan yang tepat.

3. **Service Layer (Data Preparation):** `HTMLProviderService.cs` bertugas melakukan *web scraping* untuk mengambil kode mentah HTML. Setelah itu, `DomParserService.cs` bertugas membaca (*parsing*) *string* HTML tersebut dan mengubahnya menjadi objek bertipe `DomNode.cs` yang saling terhubung membentuk representasi pohon (DOM Tree).

4. **Algorithm Engine (Traversal):** Setelah pohon terbentuk, `TraversalService.cs` akan memanggil `BfsAlgorithm.cs` atau `DfsAlgorithm.cs` sesuai pilihan pengguna. Pada setiap kunjungan simpul, algoritma akan menggunakan `SelectorMatcher.cs` untuk menguji apakah simpul tersebut memenuhi kriteria pencarian CSS.

5. **Presentation Layer:** Hasil penelusuran yang berisi waktu eksekusi, *array* jalur penelusuran, dan kecocokan data dikirim kembali ke *frontend*. Komponen `TreeView.jsx` dan `ResultsPanel.jsx` kemudian merender data tersebut menjadi animasi dan log visual yang mudah dibaca.

4.3 Implementasi Fitur Utama

4.3.1 Mekanisme Scraping dan Parsing HTML

Proses pengambilan dokumen HTML dari suatu URL (web scraping) dilakukan secara mandiri menggunakan kelas bawaan C# `System.Net.Http.HttpClient` yang dibungkus di dalam kelas `HtmlProviderService`. Beberapa mekanisme penting yang diterapkan pada proses scraping ini antara lain:

1. Manipulasi User-Agent: Untuk menghindari pemblokiran oleh server web yang menolak permintaan dari bot atau skrip otomatis, instansiasi `HttpClient` disuntikkan dengan header *User-Agent* yang mensimulasikan peramban (browser) modern (misal: Mozilla/5.0).
2. Pemrosesan Asinkron: Pengambilan string HTML menggunakan metode `GetStringAsync` yang dijalankan secara asinkron (`Task<string>`). Pendekatan ini memastikan alur eksekusi aplikasi (thread utama) tidak terblokir selama proses I/O bound saat menunggu respons dari jaringan.
3. Mekanisme Fallback (Fleksibilitas Input): Metode `GetHtmlAsync` dirancang untuk menerima dua jenis masukan: `url` dan `rawHtml`. Jika pengguna memasukkan HTML mentah secara langsung, sistem akan memprioritaskan teks tersebut dan memotong proses request jaringan. Hal ini sangat berguna untuk keperluan pengujian lokal (testing).

HTML mentah yang didapatkan diproses menjadi objek `DomNode` yang memiliki hierarki *parent-child* menggunakan kombinasi struktur data Stack. Berikut adalah kode sumber fitur HTML Scrapping yang dibuat:

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

namespace backEnd.services {
    public class HtmlProviderService {
        private readonly HttpClient client;

        public HtmlProviderService() {
            client = new HttpClient();
            client.DefaultRequestHeaders.Add("User-Agent", "
                Mozilla/5.0 (Windows NT 10.0; Win64; x64)");
        }

        public async Task<string> GetHtmlAsync(string? url,
            string? rawHtml) {
            if (!string.IsNullOrEmpty(rawHtml)) return rawHtml;

            if (!string.IsNullOrEmpty(url)) {
```

```
                    return await client.GetStringAsync(url);
                }

                throw new Exception("URL atau Raw HTML harus diisi.");
            }
        }
    }
```

Listing 1: HTMLProvider.Service.cs

Proses parsing HTML dilakukan secara manual menggunakan teknik string scanning. Sebuah variabel pos bergerak dari kiri ke kanan sepanjang string HTML, mencari karakter < sebagai penanda awal tag menggunakan IndexOf. Teks murni yang ditemukan di antara dua tag tidak digabungkan ke node parent, melainkan diinstansiasi sebagai node bertipe #text yang menjadi child eksplisit — mempertahankan posisi teks yang akurat di dalam tree. Pembangunan pohon dilakukan secara iteratif menggunakan Stack<DomNode>:

1. Komentar <!– –> dan deklarasi `<!DOCTYPE>` dideteksi dan dilewati tanpa membentuk node.
2. Ketika tag pembuka ditemukan, DomNode baru diinstansiasi. Node di puncak stack (`Peek()`) bertindak sebagai parent. Seluruh atribut, termasuk id, class, dan atribut lainnya seperti type, href, disabled, diekstraksi oleh `ParseAttributesManual` dan disimpan ke `node.Attributes`. Atribut id dan class juga disimpan ke field khusus `ElementId` dan `Classes`. Tabel `Binary Lifting Up[20]` dibangun langsung saat node dibuat untuk keperluan LCA.
3. Tag *self-closing* (void elements seperti `<img>`, `<br>`, `<input>`) dan tag dengan penutup eksplisit /> tidak di-push ke stack.
4. Ketika tag penutup ditemukan, parser menggunakan strategi pop-until-match, pop stack sampai menemukan tag yang cocok, menangani HTML yang tidak sempurna seperti tag yang lupa ditutup.

Berikut adalah kode sumber fitur HTML Parsing yang dibuat:

```
using backEnd.models;
using System;
using System.Collections.Generic;
using System.Linq;

namespace backEnd.services {
    public class DomParserService {
        private int nodeCounter_ = 0;

        private static readonly HashSet<string> VoidElements = new
            HashSet<string>(StringComparer.OrdinalIgnoreCase) {
             "area", "base", "br", "col", "embed", "hr", "img", "input
                ",
```

```
            "link", "meta", "param", "source", "track", "wbr"
        };

        public DomNode ParseAndPrepareLCA(string html) {
            nodeCounter_ = 0;
            if (string.IsNullOrWhiteSpace(html)) return null;

            var stack = new Stack<DomNode>();
            DomNode root = null;
            int pos = 0;

            while (pos < html.Length) {
                int nextOpen = html.IndexOf('<', pos);

                if (nextOpen == -1) {
                    AddTextNode(html.Substring(pos).Trim(), stack);
                    break;
                }

                if (nextOpen > pos) {
                    AddTextNode(html.Substring(pos, nextOpen - pos).
                        Trim(), stack);
                }

                pos = nextOpen;

                if (pos + 4 <= html.Length && html.Substring(pos, 4)
                    == "<!--") {
                    int commentEnd = html.IndexOf("-->", pos + 4);
                    pos = commentEnd != -1 ? commentEnd + 3 : html.
                        Length;
                    continue;
                }

                if (pos + 2 <= html.Length && html.Substring(pos, 2)
                    == "<!") {
                    int doctypeEnd = html.IndexOf('>', pos + 2);
                    pos = doctypeEnd != -1 ? doctypeEnd + 1 : html.
                        Length;
                    continue;
                }

                bool isClosingTag = false;
                int tagStart = pos + 1;
                if (tagStart < html.Length && html[tagStart] == '/')
                    {
                    isClosingTag = true;
                    tagStart++;
                }
```

```
                int tagEndIndex = html.IndexOf('>', tagStart);
                if (tagEndIndex == -1) break;

                string insideTag = html.Substring(tagStart,
                    tagEndIndex - tagStart).Trim();
                pos = tagEndIndex + 1;

                if (string.IsNullOrWhiteSpace(insideTag)) continue;

                bool isSelfClosingExplicit = false;
                if (insideTag.EndsWith("/")) {
                    isSelfClosingExplicit = true;
                    insideTag = insideTag.Substring(0, insideTag.
                        Length - 1).Trim();
                }

                string tagName;
                string attributesStr = "";

                int firstSpace = -1;
                for (int i = 0; i < insideTag.Length; i++) {
                    if (char.IsWhiteSpace(insideTag[i])) {
                        firstSpace = i;
                        break;
                    }
                }

                if (firstSpace == -1) {
                    tagName = insideTag;
                } else {
                    tagName = insideTag.Substring(0, firstSpace);
                    attributesStr = insideTag.Substring(firstSpace +
                        1);
                }

                tagName = tagName.ToLowerInvariant();
                bool isSelfClosing = isSelfClosingExplicit ||
                    VoidElements.Contains(tagName);

                if (isClosingTag) {
                    while (stack.Count > 0) {
                        var popped = stack.Pop();
                        if (string.Equals(popped.TagName, tagName,
                            StringComparison.OrdinalIgnoreCase))
                        break;
                    }
                    continue;
                }
```

```

                var parent = stack.Count > 0 ? stack.Peek() : null;
                int depth = parent != null ? parent.Depth + 1 : 0;

                var domNode = new DomNode {
                    Id = $"node-{++nodeCounter_}",
                    TagName = tagName,
                    Depth = depth,
                    IsTextNode = false
                };

                ParseAttributesManual(attributesStr, domNode);

                domNode.Up[0] = parent ?? domNode;
                for (int i = 1; i < 20; i++) {
                    domNode.Up[i] = domNode.Up[i - 1].Up[i - 1];
                }

                if (parent != null) {
                    parent.Children.Add(domNode);
                } else if (root == null) {
                    root = domNode;
                }

                if (!isSelfClosing) {
                    stack.Push(domNode);
                }
            }

            return root;
        }

        private void AddTextNode(string text, Stack<DomNode> stack) {
            if (string.IsNullOrEmpty(text) || stack.Count == 0)
               return;

            var parent = stack.Peek();

            var textNode = new DomNode {
                Id = $"node-{++nodeCounter_}",
                TagName = "#text",
                IsTextNode = true,
                TextContent = text,
                Depth = parent.Depth + 1
            };

            textNode.Up[0] = parent;
            for (int i = 1; i < 20; i++) {
                textNode.Up[i] = textNode.Up[i - 1].Up[i - 1];
```

```
            }

            parent.Children.Add(textNode);
        }

        private void ParseAttributesManual(string attrStr, DomNode
            node) {
            if (string.IsNullOrWhiteSpace(attrStr)) return;

            int pos = 0;
            while (pos < attrStr.Length) {
                while (pos < attrStr.Length && char.IsWhiteSpace(
                    attrStr[pos])) pos++;
                if (pos >= attrStr.Length) break;

                int keyStart = pos;
                while (pos < attrStr.Length && attrStr[pos] != '=' &&
                    !char.IsWhiteSpace(attrStr[pos])) pos++;
                string key = attrStr.Substring(keyStart, pos -
                    keyStart).ToLowerInvariant();

                if (string.IsNullOrEmpty(key)) { pos++; continue; }

                while (pos < attrStr.Length && char.IsWhiteSpace(
                    attrStr[pos])) pos++;

                if (pos >= attrStr.Length || attrStr[pos] != '=') {
                    node.Attributes[key] = "";
                    continue;
                }
                pos++;

                while (pos < attrStr.Length && char.IsWhiteSpace(
                    attrStr[pos])) pos++;
                if (pos >= attrStr.Length) break;

                string value = "";
                char quote = attrStr[pos];
                if (quote == '"' || quote == '\'') {
                    pos++;
                    int valStart = pos;
                    while (pos < attrStr.Length && attrStr[pos] !=
                        quote) pos++;
                    value = attrStr.Substring(valStart, pos -
                        valStart);
                    if (pos < attrStr.Length) pos++;
                } else {
                    int valStart = pos;
                    while (pos < attrStr.Length && !char.IsWhiteSpace
```

```
                        (attrStr[pos])) pos++;
                    value = attrStr.Substring(valStart, pos - valStart);
                }

                if (key == "id") {
                    node.ElementId = value;
                } else if (key == "class") {
                    node.Classes = value.Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries).ToList();
                }

                node.Attributes[key] = value;
            }
        }

        public DomNode GetLCA(DomNode u, DomNode v) {
            if (u == null || v == null) return null;

            if (u.IsTextNode) u = u.Up[0];
            if (v.IsTextNode) v = v.Up[0];

            if (u.Depth < v.Depth) (u, v) = (v, u);

            for (int i = 19; i >= 0; i--)
            if (u.Depth - (1 << i) >= v.Depth)
            u = u.Up[i];

            if (u == v) return u;

            for (int i = 19; i >= 0; i--)
            if (u.Up[i] != v.Up[i]) {
                u = u.Up[i];
                v = v.Up[i];
            }

            return u.Up[0];
        }
    }
}
```

Listing 2: DomParserService.cs

4.3.2 Algoritma Traversal (BFS dan DFS)

Penelusuran pohon DOM (DOM Traversal) diimplementasikan secara terpusat melalui kelas `TraversalService` yang mendelegasikan tugas ke kelas spesifik `BfsAlgorithm` dan

`DfsAlgorithm`. Berikut adalah kode sumber algoritma traversal yang dibuat:

```
using System;
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Linq;
using System.Threading;
using backEnd.models;
using backEnd.algorithms;

namespace backEnd.services
{
    public class TraversalService
    {
        public SearchResponse Traverse(DomNode root, string selector, string algorithm, bool isParallel, int? maxResults)
        {
            var response = new SearchResponse { Tree = root };
            var stopwatch = Stopwatch.StartNew();

            var matchedNodes = new ConcurrentBag<DomNode>();
            var traversalPath = new ConcurrentQueue<string>();
            var traversalLog = new ConcurrentQueue<string>();

            int nodesVisited = 0;
            int maxDepth = 0;

            void LogVisit(DomNode node)
            {
                Interlocked.Increment(ref nodesVisited);

                int currentMax;
                do
                {
                    currentMax = maxDepth;
                } while (node.Depth > currentMax && Interlocked.CompareExchange(ref maxDepth, node.Depth, currentMax) != currentMax);

                traversalPath.Enqueue(node.Id);
                traversalLog.Enqueue($"[THREAD {Environment.CurrentManagedThreadId}] Mengunjungi <{node.TagName}> (ID: {node.Id})");

                if (SelectorMatcher.Matches(node, selector))
                {
                    matchedNodes.Add(node);
                    traversalLog.Enqueue($"[MATCH] Menemukan <{node.TagName}> cocok dengan '{selector}'")
```

```
                                ;
                        }
                    }

                    if (algorithm.Equals("bfs", StringComparison.OrdinalIgnoreCase))
                    {
                        if (isParallel)
                        {
                            BfsAlgorithm.ExecuteParallel(root, LogVisit, maxResults, matchedNodes);
                        }
                        else
                        {
                            BfsAlgorithm.ExecuteNormal(root, LogVisit, maxResults, matchedNodes);
                        }
                    }
                    else
                    {
                        if (isParallel)
                        {
                            DfsAlgorithm.ExecuteParallel(root, LogVisit, maxResults, matchedNodes);
                        }
                        else
                        {
                            DfsAlgorithm.ExecuteNormal(root, LogVisit, maxResults, matchedNodes);
                        }
                    }

                    stopwatch.Stop();

                    response.MatchedNodeIds = matchedNodes.Select(n => n.Id).ToList();
                    response.TraversalPath = traversalPath.ToList();
                    response.TraversalLog = traversalLog.ToList();
                    response.NodesVisited = nodesVisited;
                    response.TimeTakenMs = stopwatch.Elapsed.TotalMilliseconds;
                    response.MaxDepth = maxDepth;

                    return response;
                }
            }
        }
```

Listing 3: TraversalService.cs

1. Breadth-First Search (BFS) Versi normal (sekuensial) menggunakan `Queue<DomNode>` bawaan C# untuk menelusuri graf secara melebar. Node diekstraksi, dicocokkan dengan kriteria *selector*, dan anak-anaknya dimasukkan ke antrean dari kiri ke kanan. Berikut adalah kode sumber algoritma BFS:

```
    public static void ExecuteNormal(DomNode root, Action<
       DomNode> visitAction, int? maxResults, ConcurrentBag<
       DomNode> matches) {
        if (root == null) return;

        var queue = new Queue<DomNode>();
        queue.Enqueue(root);

        while (queue.Count > 0) {
            if (maxResults.HasValue && matches.Count >=
               maxResults.Value)
            break;

            var node = queue.Dequeue();

            visitAction(node);

            if (node.Children != null) {
                foreach (var child in node.Children) {
                    queue.Enqueue(child);
                }
            }
        }
    }
```

Listing 4: BfsAlgorithm.cs

2. Depth-First Search (DFS) Versi normal (sekuensial) menggunakan `Stack<DomNode>`. Agar urutan penelusuran identik dengan posisi visual DOM (kiri ke kanan), anak-anak (*children*) dari sebuah node dimasukkan ke dalam stack secara terbalik (dari indeks terbesar ke terkecil) menggunakan perulangan *for* mundur. Berikut adalah kode sumber algoritma DFS:

```
    public static void ExecuteNormal(DomNode root, Action<
       DomNode> visitAction, int? maxResults, ConcurrentBag<
       DomNode> matches)
    {
        if (root == null) return;

        var stack = new Stack<DomNode>();
        stack.Push(root);

        while (stack.Count > 0)
        {
```

```
            if (maxResults.HasValue && matches.Count >= maxResults.Value)
            break;

            var node = stack.Pop();

            visitAction(node);

            if (node.Children != null)
            {
                for (int i = node.Children.Count - 1; i >= 0; i--)
                {
                    stack.Push(node.Children[i]);
                }
            }
        }
    }
```

Listing 5: DfsAlgorithm.cs

4.3.3 Evaluasi CSS Selector

Logika pencocokan node dikelola oleh kelas SelectorMatcher. Parser selector mendukung Selector Dasar (tag, #id, .class), Atribut Selector ([attr], [attr="value"]), Selector Majemuk, dan Combinator (>, , +,). Evaluasi dilakukan secara struktural Bottom-Up.

1. Single/Compound Selector (MatchesSingle): Selector diparsing menggunakan loop berbasis index pos yang membaca karakter per karakter. Karakter #, ., [, dan : menjadi penanda pergantian mode. Blok [...] dibaca secara utuh — isinya dipisahkan berdasarkan ada tidaknya = untuk membedakan selector keberadaan atribut ([disabled]) dan selector nilai atribut ([type="text"]). Pengecekan dilakukan terhadap `node.TagName`, `node.ElementId`, `node.Classes` (menggunakan Contains per class), dan `node.Attributes` (Dictionary semua atribut). Pseudo-class seperti :hover dikenali dan dilewati tanpa menyebabkan error.

2. Combinator (EvaluateCombinator): Selector dipecah menjadi token oleh `TokenizeSelector` yang sadar blok [...] — tokenizer melompati isi [...] secara utuh agar tidak terpotong di tengah. Pengecekan dimulai dari token paling kanan (elemen target). Jika cocok, algoritma memanggil `GetParent()` atau `GetPreviousSibling()` sesuai jenis combinator. `GetPreviousSibling()` secara otomatis melewati #text node karena combinator + dan hanya berlaku untuk element node.

Berikut adalah kode sumber CSS *selector* yang dibuat:

```
    using backEnd.models;
    using System;
```

```
using System.Collections.Generic;
using System.Linq;

namespace backEnd.algorithms {
    public static class SelectorMatcher {

        public static bool Matches(DomNode node, string selector) {
            if (node.IsTextNode) return false;
            if (string.IsNullOrWhiteSpace(selector)) return false;

            selector = selector.Trim();

            bool hasCombinator = false;
            foreach (char c in selector) {
                if (char.IsWhiteSpace(c) || c == '>' || c == '+' || c == '~') {
                    hasCombinator = true;
                    break;
                }
            }

            return hasCombinator ? EvaluateCombinator(node, selector) : MatchesSingle(node, selector);
        }

        public static bool MatchesSingle(DomNode node, string selector) {
            if (node.IsTextNode) return false;
            if (selector == "*") return true;

            string? tag = null;
            string? id = null;
            List<string> classes = new List<string>();
            List<(string key, string? value)> attrSelectors = new List<(string, string?)>();

            int pos = 0;
            while (pos < selector.Length) {
                char c = selector[pos];

                if (c == '#') {
                    pos++;
                    int start = pos;
                    while (pos < selector.Length && selector[pos] != '.' &&
                    selector[pos] != '#' && selector[pos] != '[' &&
```

```
                    selector[pos] != ':') pos++;
                    id = selector.Substring(start, pos - start);

                } else if (c == '.') {
                    pos++;
                    int start = pos;
                    while (pos < selector.Length && selector[pos] != '.' &&
                    selector[pos] != '#' && selector[pos] != '[' &&
                    selector[pos] != ':') pos++;
                    classes.Add(selector.Substring(start, pos - start));

                } else if (c == '[') {
                    pos++;
                    int start = pos;
                    while (pos < selector.Length && selector[pos] != ']') pos++;
                    string attrExpr = selector.Substring(start, pos - start);
                    pos++;

                    int eqIndex = attrExpr.IndexOf('=');
                    if (eqIndex == -1) {
                        attrSelectors.Add((attrExpr.Trim().ToLowerInvariant(), null));
                    } else {
                        string attrKey = attrExpr.Substring(0, eqIndex).Trim().ToLowerInvariant();
                        string attrVal = attrExpr.Substring(eqIndex + 1).Trim().Trim('"', '\'');
                        attrSelectors.Add((attrKey, attrVal));
                    }

                } else if (c == ':') {
                    pos++;
                    while (pos < selector.Length && selector[pos] != '.' &&
                    selector[pos] != '#' && selector[pos] != '[' &&
                    selector[pos] != ':') pos++;

                } else {
                    int start = pos;
                    while (pos < selector.Length && selector[pos] != '.' &&
                    selector[pos] != '#' && selector[pos] != '[' &&
```

```
                        selector[pos] != ':') pos++;
                        string tagStr = selector.Substring(start, pos - start);
                        if (!string.IsNullOrEmpty(tagStr)) tag = tagStr;
                    }
                }

                if (tag != null && !string.Equals(node.TagName, tag, StringComparison.OrdinalIgnoreCase))
                return false;

                if (id != null && !string.Equals(node.ElementId, id, StringComparison.OrdinalIgnoreCase))
                return false;

                if (classes.Any()) {
                    if (node.Classes == null || !node.Classes.Any()) return false;
                    foreach (var cls in classes)
                    if (!node.Classes.Contains(cls, StringComparer.OrdinalIgnoreCase))
                    return false;
                }

                if (attrSelectors.Any()) {
                    foreach (var (attrKey, attrVal) in attrSelectors) {
                        if (!node.Attributes.ContainsKey(attrKey))
                        return false;

                        if (attrVal != null && !string.Equals(node.Attributes[attrKey], attrVal, StringComparison.OrdinalIgnoreCase))
                        return false;
                    }
                }

                return true;
            }

            private static bool EvaluateCombinator(DomNode node, string selector) {
                var tokens = TokenizeSelector(selector);
                if (tokens.Count == 0) return false;

                int i = tokens.Count - 1;
                DomNode? currentNode = node;

```

```
                if (!MatchesSingle(currentNode, tokens[i])) return false;
                i--;

                while (i >= 0) {
                    string token = tokens[i];
                    bool isCombinator = token == ">" || token == "+" || token == "~" || token == " ";

                    string combinator;
                    string targetSelector;

                    if (isCombinator) {
                        combinator = token;
                        i--;
                        if (i < 0) return false;
                        targetSelector = tokens[i];
                    } else {
                        combinator = " ";
                        targetSelector = token;
                    }

                    if (combinator == ">") {
                        currentNode = GetParent(currentNode);
                        if (currentNode == null || !MatchesSingle(currentNode, targetSelector)) return false;
                    } else if (combinator == " ") {
                        bool found = false;
                        currentNode = GetParent(currentNode);
                        while (currentNode != null) {
                            if (MatchesSingle(currentNode, targetSelector)) { found = true; break; }
                            currentNode = GetParent(currentNode);
                        }
                        if (!found) return false;
                    } else if (combinator == "+") {
                        currentNode = GetPreviousSibling(currentNode);
                        if (currentNode == null || !MatchesSingle(currentNode, targetSelector)) return false;
                    } else if (combinator == "~") {
                        bool found = false;
                        currentNode = GetPreviousSibling(currentNode);
                        while (currentNode != null) {
                            if (MatchesSingle(currentNode,
```

```
                                targetSelector)) { found = true; break; }
                            currentNode = GetPreviousSibling(currentNode);
                        }
                        if (!found) return false;
                    }

                    i--;
                }

                return true;
            }

            private static List<string> TokenizeSelector(string selector) {
                var tokens = new List<string>();
                int pos = 0;

                while (pos < selector.Length) {
                    while (pos < selector.Length && char.IsWhiteSpace(selector[pos])) pos++;
                    if (pos >= selector.Length) break;

                    char c = selector[pos];
                    if (c == '>' || c == '+' || c == '~') {
                        if (tokens.Count > 0 && tokens[tokens.Count - 1] == " ")
                        tokens.RemoveAt(tokens.Count - 1);
                        tokens.Add(c.ToString());
                        pos++;
                    } else {
                        int start = pos;
                        while (pos < selector.Length && !char.IsWhiteSpace(selector[pos]) &&
                        selector[pos] != '>' && selector[pos] != '+' && selector[pos] != '~') {
                            if (selector[pos] == '[') {
                                while (pos < selector.Length && selector[pos] != ']') pos++;
                                if (pos < selector.Length) pos++;
                            } else {
                                pos++;
                            }
                        }
                        tokens.Add(selector.Substring(start, pos - start));

                        bool hadSpace = false;
```

```
                        while (pos < selector.Length && char.IsWhiteSpace(selector[pos])) {
                            pos++;
                            hadSpace = true;
                        }

                        if (hadSpace && pos < selector.Length && selector[pos] != '>' && selector[pos] != '+' && selector[pos] != '~') {
                            tokens.Add(" ");
                        }
                    }
                }
                return tokens;
            }

            private static DomNode? GetParent(DomNode? node) {
                if (node != null && node.Up != null && node.Up[0] != node)
                return node.Up[0];
                return null;
            }

            private static DomNode? GetPreviousSibling(DomNode? node) {
                var parent = GetParent(node);
                if (parent == null) return null;
                int index = parent.Children.IndexOf(node!);
                for (int i = index - 1; i >= 0; i--)
                if (!parent.Children[i].IsTextNode)
                return parent.Children[i];
                return null;
            }
        }
    }
```

Listing 6: SelectorMatcher.cs

4.4 Implementasi Fitur Bonus

4.4.1 Visualisasi dan Animasi Penelusuran

Untuk menyediakan pengalaman visual secara real-time di antarmuka (frontend), state pergerakan algoritma direkam ke dalam dua koleksi aman-utas (thread-safe) yaitu `ConcurrentQueue<string> traversalPath` dan `traversalLog`.

Metode aksi `LogVisit(DomNode node)` disisipkan ke dalam algoritma traversal (sebagai fungsi lambda/delegasi). Setiap kali suatu node dikunjungi, ID dari node terse-

but dimasukkan ke dalam `traversalPath`, dan pesan deskriptif dimasukkan ke dalam `traversalLog`. Hasil ini dibungkus ke dalam `SearchResponse` dan dikembalikan ke *frontend*. Di sisi *frontend*, array ID path ini digunakan untuk menyalakan indikator visual pada representasi pohon graf setahap demi setahap secara berurutan.

4.4.2 Multithreading pada Traversal

Untuk memproses struktur DOM berskala masif dengan cepat, diterapkan pemrograman paralel menggunakan pustaka *Task Parallel Library* (TPL) bawaan C#. Agar terhindar dari kondisi balapan (*race condition*), semua hasil kecocokan disimpan menggunakan `ConcurrentBag<DomNode>`.

1. Paralelisasi BFS: Algoritma membaca jumlah node pada level saat ini (`levelSize`), menyimpannya ke dalam koleksi lokal currentLevelNodes, lalu memproses semua node di level yang sama secara bersamaan menggunakan `Parallel.ForEach`. Anak-anak dari level tersebut dimasukkan ke dalam `ConcurrentQueue` tunggal untuk level berikutnya. Berikut adalah kode sumber algotima BFS dengan penerapan *multithreading*:

```
        public static void ExecuteParallel(DomNode root, Action<DomNode> visitAction, int? maxResults, ConcurrentBag<DomNode> matches) {
            var queue = new ConcurrentQueue<DomNode>();
            queue.Enqueue(root);

            while (!queue.IsEmpty) {
                if (maxResults.HasValue && matches.Count >= maxResults.Value) break;

                int levelSize = queue.Count;
                var currentLevelNodes = new List<DomNode>(levelSize);

                for (int i = 0; i < levelSize; i++) {
                    if (queue.TryDequeue(out var node))
                    currentLevelNodes.Add(node);
                }
                Parallel.ForEach(currentLevelNodes, (node, state) => {
                    if (state.IsStopped) return;

                    if (maxResults.HasValue && matches.Count >= maxResults.Value) {
                        state.Stop();
                        return;
                    }

                    visitAction(node);
```

```

                    if (maxResults.HasValue && matches.Count >= maxResults.Value)
                    state.Stop();

                    foreach (var child in node.Children)
                    queue.Enqueue(child);
                });
            }
        }
```

Listing 7: BfsAlgorithm.cs

2. Paralelisasi DFS: Menggunakan strategi delegasi per cabang. Akar (root) dievaluasi terlebih dahulu, kemudian algoritma memanggil `Parallel.ForEach` terhadap kumpulan `root.Children`. Setiap thread secara independen membuat instance `Stack<DomNode>` lokal dan menelusuri sub-pohonnya masing-masing tanpa harus mengunci koleksi global. Berikut adalah kode sumber algotima DFS dengan penerapan *multithreading*:

```
        public static void ExecuteParallel(DomNode root, Action<DomNode> visitAction, int? maxResults, ConcurrentBag<DomNode> matches)
        {
            if (root == null) return;

            visitAction(root);

            if (root.Children == null || root.Children.Count == 0) return;

            Parallel.ForEach(root.Children, (child, state) =>
            {
                if (maxResults.HasValue && matches.Count >= maxResults.Value) { state.Stop(); return; }

                var stack = new Stack<DomNode>();
                stack.Push(child);

                while (stack.Count > 0)
                {
                    if (maxResults.HasValue && matches.Count >= maxResults.Value) break;

                    var node = stack.Pop();
                    visitAction(node);

                    if (node.Children != null)
                    for (int i = node.Children.Count - 1; i >= 0; i--)
```

```
                    stack.Push(node.Children[i]);
                }
            });
        }
```

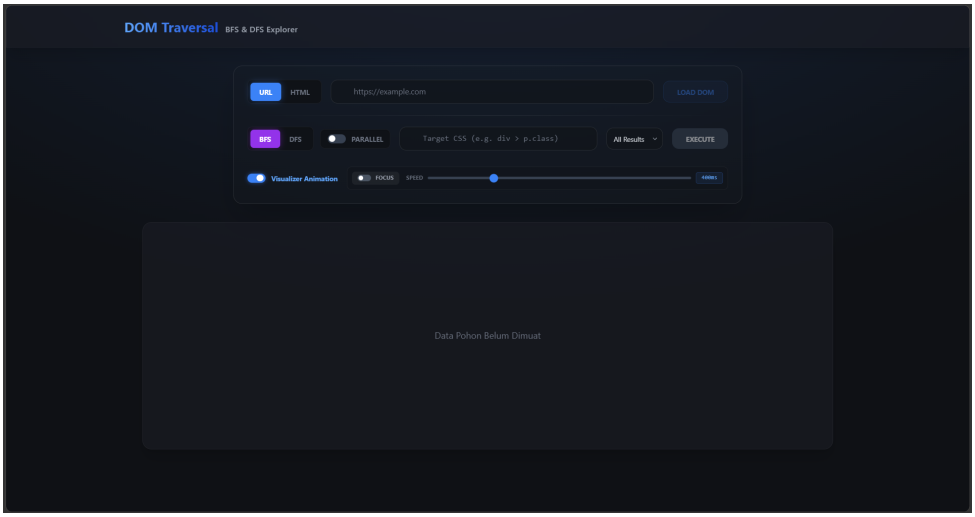
Listing 8: DfsAlgorithm.cs

4.4.3 Lowest Common Ancestor (LCA) Binary Lifting

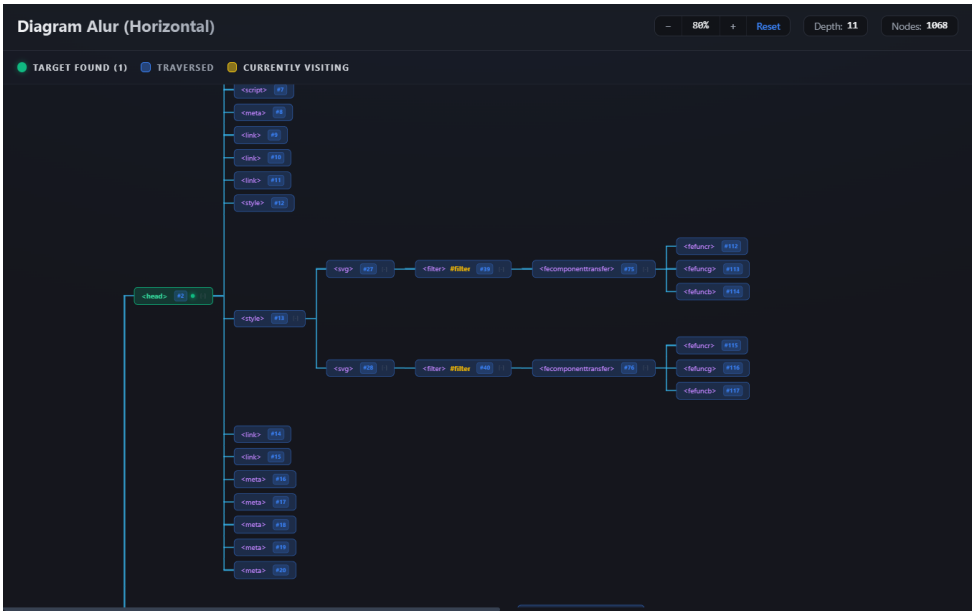
Teknik *Binary Lifting* diimplementasikan untuk memberikan waktu query tercepat $O(\log N)$ dalam penentuan leluhur terdekat dari dua buah node. Hal ini sangat krusial pada penelusuran relasi pohon DOM yang berakar sangat dalam.

1. Fase Komputasi Awal (Pre-computation): Dilakukan secara bersamaan di dalam metode parsing HTML. Instansiasi setiap `DomNode` secara dinamis menyiapkan array `Up` dengan alokasi 20 indeks. Relasi leluhur dihitung menggunakan relasi eksponensial: leluhur pada lompatan 2^i adalah leluhur dari node yang berada pada lompatan 2^{i-1} sejauh 2^{i-1} tingkat lagi (`domNode.Up[i] = domNode.Up[i - 1].Up[i - 1]`).
2. Fase Evaluasi (Query): Algoritma pada `GetLCA(u, v)` terlebih dahulu menyelaraskan tingkat kedalaman (Depth) antara node `u` dan `v` dengan memanfaatkan operasi bitwise ($1 << i$). Apabila level sudah setara dan target belum bertemu, algoritma memindahkan pointer `u` dan `v` secara serentak melompat ke atas secara eksponensial hingga menemukan persimpangan leluhur yang identik.

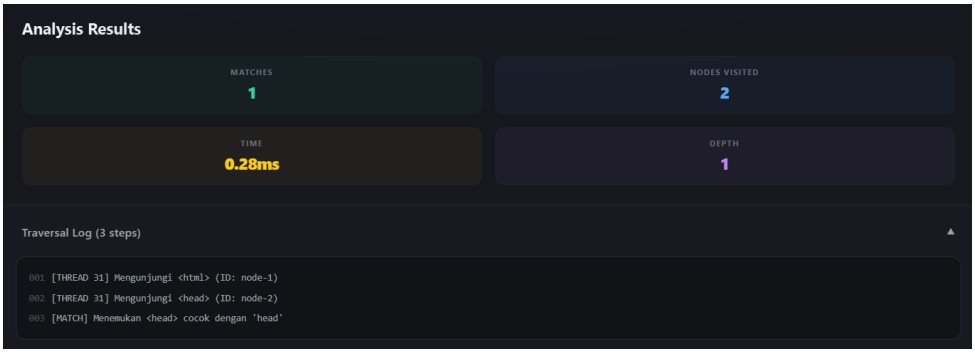
4.5 Antarmuka Pengguna (UI)



Gambar 6: Tampilan Utama Aplikasi (Input URL dan Selector)



Gambar 7: Visualisasi Pohon DOM dan Jalur Traversal



Gambar 8: Panel Log Traversal

Pada program yang kami buat, terdapat beberapa fitur yang mendukung pencarian (*traversal*) pada kode HTML. User dapat memilih apakah ia ingin mendapatkan kode sumber HTML dari pranala di internet melalui *Web Scrapping* atau user juga dapat memasukkan kode program HTML yang dimiliki dengan mengganti opsi menjadi `HTML`. Selain itu user juga dapat memilih algoritma apa yang akan digunakan untuk melakukan traversal yaitu BFS atau DFS. Selain itu user juga dapat mengaktifkan tombol `parallel` untuk mengeksekusi traversal dengan *multithreading*. Terdapat juga field untuk memasukkan CSS selector yang akan dicari. User juga dapat membatasi pencarian dengan memilih opsi `Top-n`, sehingga traversal akan berhenti secara paksa jika sudah ketemu n node yang sesuai dengan selector. Tombol visualisasi animasi dapat diaktifkan dengan kecepatan animasi yang juga dapat diatur untuk menampilkan animasi visualisasi pencarian/traversal yang dialakukan oleh program. Terdapat juga panel log yang menampilkan statistik traversal serta log traversal yang dilakukan.

4.6 Hasil Pengujian dan Analisis

4.6.1 Skenario Pengujian

Pengujian fungsionalitas utama aplikasi dibagi menjadi dua skenario utama untuk mengevaluasi akurasi parser dan keandalan algoritma penelusuran (BFS/DFS):

1. Pengujian Lokal (Mock HTML): Menggunakan dokumen HTML buatan dengan struktur yang disengaja untuk menguji logika kombinasi CSS Selector yang kompleks (khususnya combinator seperti >, +, , dan spasi).
2. Pengujian *Web Scraping*: Melakukan *scraping* langsung ke situs web berskala besar yang memiliki pohon DOM dalam dan kompleks untuk menguji kinerja multithreading dan ketahanan aplikasi. Situs yang dipilih adalah Halaman Utama Wikipedia Bahasa Indonesia (https://id.wikipedia.org/wiki/Halaman_Utama).

Pengujian dilakukan dengan memasukkan CSS Selector, lalu memvalidasi apakah `MatchedNodeIds` pada `SearchResponse` mengembalikan elemen yang sesuai dengan ekspektasi.

4.6.2 Kasus Uji 1 (Mock HTML)

Pada skenario ini, HTML mentah berikut disuntikkan secara langsung (raw HTML) ke dalam sistem tanpa melewati proses network request. Berikut adalah kode sumber yang akan digunakan untuk pengujian:

```html
<div id="wrapper">
  <section class="content container">
  <h1 class="title">Pengujian DOM</h1>
  <p class="text">Paragraf pertama.</p>
  <div class="box">
    <span class="badge">Target A</span>
    <p>Paragraf di dalam box.</p>
    <span class="badge highlight">Target B</span>
  </div>
  <p class="text highlight">Paragraf kedua.</p>
  <span class="info">Target C</span>
  <span class="info">Target D</span>
  </section>
</div>
```

Listing 9: mock HTML

Berikut adalah tabel kasus uji yang akan dilakukan:

No	CSS Selector	Penjelasan Logika	Expected Output (TextContent)
1	`div#wrapper section.container > p.text.highlight`	Mencari elemen <p> dengan *multiple class* (`text` dan `highlight`), yang merupakan anak langsung (>) dari <section> berkelas `container`, di dalam `div#wrapper`.	`Paragraf kedua.`
2	`div.box p + span.badge`	Menguji *Adjacent Sibling* (+). Mencari `span.badge` yang posisinya persis bersebelahan setelah <p> di dalam `div.box`.	`Target B`
3	`h1.title ~ span`	Menguji *General Sibling* (~). Mencari semua yang merupakan saudara (memiliki *parent* yang sama) dan letaknya setelah <h1>.	`Target C`, `Target D`
4	`div#wrapper .box > span`	Gabungan *Descendant* (spasi) dan *Child* (>). Mencari yang merupakan anak langsung dari `.box`, di mana `.box` berada di bawah `div#wrapper`.	`Target A`, `Target B`

Tabel 3: Kasus Uji Skenario 1: Lokal (Mock HTML)

4.6.3 Kasus Uji 2 (Web Scrapping)

Skenario ini menggunakan `HtmlProviderService` untuk mengunduh HTML dari `https://id.wikipedia.org/wiki/Halaman_Utama`. Pohon DOM Wikipedia sangat besar dan bersarang (*nested*), menjadikannya dataset yang sempurna untuk menguji parser kustom buatan sendiri. Berikut adalah tabel kasus uji yang akan dilakukan:

Tabel 4: Kasus Uji Skenario 2: Web Scraping (Wikipedia)

No	CSS Selector	Penjelasan Logika	Expected Output (Gambaran Hasil)
1	`h1.firstHeading`	Mencari elemen judul utama halaman (`h1`) yang memiliki kelas `firstHeading`.	Teks "Halaman Utama"
2	`body a`	Mencari semua tag tautan (`a`) yang berada di dalam `body`. Ini akan menguji kemampuan penelusuran *descendant*.	Daftar berbagai tautan navigasi atau artikel.
3	`div > p`	Mencari paragraf (`p`) yang merupakan anak langsung dari sebuah `div`. Wikipedia memiliki ribuan struktur seperti ini.	Potongan teks paragraf artikel.

4.6.4 Analisis Performa BFS vs DFS

1. Skenario Kasus Uji 1 (tanpa *multithreading*)

Metrik	**TC 1**		**TC 2**		**TC 3**		**TC 4**	
	BFS	**DFS**	**BFS**	**DFS**	**BFS**	**DFS**	**BFS**	**DFS**
Matches	1	1	1	1	2	2	2	2
Waktu (ms)	0.08	0.1	0.05	0.06	0.06	0.06	0.06	0.05
Node Dikunjungi	11	11	11	11	11	11	11	11
Kedalaman Maksimum	3	3	3	3	3	3	3	3
Jalur Ditemukan	✓	✓	✓	✓	✓	✓	✓	✓

Tabel 5: Hasil Eksekusi Skenario 1 (Lokal)

2. Skenario Kasus Uji 1 (dengan *multithreading*)

Metrik	**TC 1**		**TC 2**		**TC 3**		**TC 4**	
	BFS	**DFS**	**BFS**	**DFS**	**BFS**	**DFS**	**BFS**	**DFS**
Matches	1	1	1	1	2	2	2	2
Waktu (ms)	0.14	0.13	0.12	0.1	0.08	0.09	0.11	0.09
Node Dikunjungi	11	11	11	11	11	11	11	11
Kedalaman Maksimum	3	3	3	3	3	3	3	3
Jalur Ditemukan	✓	✓	✓	✓	✓	✓	✓	✓

Tabel 6: Hasil Eksekusi Skenario 1 (Lokal) *multithreading*

3. Skenario Kasus Uji 2 (tanpa *multithreading*)

Metrik	**TC 1**		**TC 2**		**TC 3**	
	BFS	**DFS**	**BFS**	**DFS**	**BFS**	**DFS**
Matches	1	1	127	127	3	3
Waktu (ms)	0.79	0.64	1.07	1.05	0.6	0.6
Node Dikunjungi	771	771	771	771	771	771
Kedalaman Maksimum	20	20	20	20	20	20
Jalur Ditemukan	✓	✓	✓	✓	✓	✓

Tabel 7: Hasil Eksekusi Skenario 2 (Lokal)

4. Skenario Kasus Uji 2 (dengan *multithreading*)

Metrik	**TC 1**		**TC 2**		**TC 3**	
	BFS	**DFS**	**BFS**	**DFS**	**BFS**	**DFS**
Matches	1	1	127	127	3	3
Waktu (ms)	0.71	0.55	0.72	0.68	0.53	0.42
Node Dikunjungi	771	771	771	771	771	771
Kedalaman Maksimum	20	20	20	20	20	20
Jalur Ditemukan	✓	✓	✓	✓	✓	✓

Tabel 8: Hasil Eksekusi Skenario 2 (Lokal) *multithreading*

Berdasarkan hasil pengujian yang telah dilakukan, beikut adalah analisis yang dilakukan didapatkan:

1. Perbandingan kecepatan BFS dan DFS: Perbandingan kecepatan pemrosesan yang dilakukan oleh algoritma BFS dan DFS bergantung pada struktur *tree*. Jika tree merentang ke samping (banyak node dalam satu level), maka pemrosesan DFS akan diuntungkan karena ia menelusuri andak dari sebuah *node* hingga sampai node paling akhir, sedangkan BFS harus memproses node di level yang sama terlebih dahulu. Sedangkan jika sturuktur *tree* memiliki kedalaman yang sangat dalam, maka BFS akan diuntungkan. Namun, perbandingan kecepatan BFS dan DFS tidak hanya bergantung pada bentuk tree, tetapi juga pada lokasi solusi karena pada dasarnya time complexity dari keduanya sama yakni $O(u + v)$. BFS lebih optimal untuk pencarian pada level dangkal karena menelusuri per level, sedangkan DFS lebih optimal pada tree yang dalam karena langsung menelusuri hingga kedalaman maksimum.

2. Pemrosesan multithreading: Pada tabel hasil uji diperlihatkan bahwa penggunaan multithreading untuk *tree* yang sederhana seringkali lebih lambah dibandingkan dengan pemrosesan sekuensial biasa. Hal ini dapat terjadi karena terdapat banyak *overhead* untuk melakukan *thread creation*, *schedulling*, dan *synchronization* oleh *worker* pada *thread*. Pada Uji Kasus 2 (*scrapping website*), didapatkan tree dengan node yang cukup banyak. Dapat diperhatikan juga bahwa pemrosesan dengan

menggunakan *multithreading* lebih cepat daripada pemrosesan biasa tanpa *multithreading*. Walaupun perbedaan tidak signifikan (hanya berbeda beberapa ms) namun hal ini menunjukkan bahwa algoritma multithreading yang diimplementasikan berjalan dengan baik.

3. Web scraping: Proses *web scrapping* yang dilakukan oleh *HTTP client* berhasil mengambil konten halaman dalam bentuk *raw HTML*, yang kemudian dapat diproses lebih lanjut untuk ekstraksi data.

BAB 5
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berikut adalah kesimpulan yang dapat diambil:

1. Dalam kasus rata-rata, tidak ada yang lebih efisien antara algoritma BFS maupun DFS karena keduanya memiliki *time complexity* yang sama yakni $O(u+v)$. Namun dalam beberapa skenario khusus yang dijelaskan sebelumnya pada bagian analisis, keduanya dapat lebih baik dibandingkan dengan satu lainnya.

2. Penggunaan *multithreading* diperuntukkan untuk memproses *tree* dengan jumlah node yang sangat banyak, misalnya ribuan hingga puluhan ribu. Dengan demikian dapat dilihat efek dari *multithreading* yang diterapkan akan berbuah signifikan dibandingkan dengan pemrosesan normal.

3. *HTML scrapping* dan *parsing* yang diimplementasikan dapat berjalan dengan baik. Hal ini ditunjukkan untuk kode HTML yang sama akan didapatkan struktur DOM tree yang sama juga setiap waktu (konsisten). Akan tetapi ketika melakukan *scrapping* pada website yang dinamis, seringkali DOM tree yang didapatkan tidak konsisten, jumlah node dapat berkurang atau bertambah sewaktu-waktu. Namun hal itu juga menunjukkan bahwa *scrapping* kita berjalan ssesuai kondisi *real time*.

5.2 Saran

Berikut adalah beberapa saran pengembangan oleh penulis:

1. Walaupun struktur LCA sudah ditambahkan dan diterapkan dalam `DOMNode` (`Up`) akan tetapi penerapan langsung untuk fitur LCA ini masih belum direalisasikan. Hal ini karena kebingungan dan ketidakmampuan penulis dalam memahami spesifikasi bonus/lanjutan yang diminta. Saran penulis adalah pengimplementasian LCA dapat diterapkan pada *frontend* dengan memberikan *field* baru untuk mengetahui node leluhur terendah yang sama dari dua node dalam *tree*.

2. Dalam melakukan parsing HTML tidak dilakukan *error checking* pada kode HTML yang di-*input*-kan. Alasan penulis melakukan hal tersebut *it's by design*. HTML parsing yang diimplementasikan oleh penulis tidak melakukan pengecekan seperti *typo* pada tag pembuka atau penutup pada kode HTML, selain itu juga tidak mengecek *mismatch* tag yang dapat terjadi pada kode HTML (e.g. `<head><h1></head></h1>`). Penulis mengasumsikan bahwa kode HTML yang di-*input* secara manual adalah kode HTML yang valid dan dapat dijalankan oleh browser. walaupun *error checking* tidak diterapkan, program yang dibuat tetaplah masih *robust* untuk semua kode HTML pada internet.

3. Untuk uji coba yang lebih komprehensif sebaiknya dilakukan uji coba juga dengan *tree* yang memiliki banyak node (ribuan hingga puluhan ribu). Hal ini dilakukan untuk mengetahui kapasitas berjalannya program yang telah dibuat serta efisiensi dari algoritma yang diimplementasikan.

A Lampiran

A.1 Tautan Repositori GitHub dan Video

Repositori GitHub: https://github.com/Alpaomega1136/Tubes2_e621.git
Video Demo: https://youtu.be/8fe-3ZK7InU

A.2 Tabel Checklist Kelengkapan

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting		✓

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

An-Dafa Anza Avansyah

M. Haris Putra S.

Raymond Jonathan Dwi Putra Julianto

Pustaka

[1] Munir, R. (2026). *Breadth First Search (BFS) dan Depth First Search (DFS) Bagian 1*. Program Studi Teknik Informatika, Institut Teknologi Bandung. Diakses dari: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-(2026)-Bagian1.pdf

[2] Munir, R. (2026). *Breadth First Search (BFS) dan Depth First Search (DFS) Bagian 2*. Program Studi Teknik Informatika, Institut Teknologi Bandung. Diakses dari: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-(2026)-Bagian2.pdf

[3] MDN Web Docs. (n.d.). *Document Object Model (DOM)*. Diakses dari: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

[4] MDN Web Docs. (n.d.). *CSS selectors*. Diakses dari: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors